**UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO**

**FACULTAD DE INGENIERÍA ELÉCTRICA,**
**ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA**
**Y DE SISTEMAS**




**INFORME DE GUÍA DE LABORATORIO N.º 07**

---

# PRIMALIDAD, GRUPOS Y ALGORITMOS PROBABILÍSTICOS

---

**Asignatura:** ALGORITMOS AVANZADOS

**Docente:** Ing. Héctor Eduardo Ugarte Rojas

**Equipo:** GRUPO 5 **Semestre:** 2026-II

**Integrantes:**

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

**Cusco – Perú**

**2026**

# Índice

**1. Trabajo preparatorio y fundamentos algebraicos** **1**

1.1. Unidades módulo $n$, función $\varphi$ y orden . . . . . 1

1.2. División de prueba, pseudoprimos y Carmichael . . . . . 1

1.3. Exponenciación binaria y Miller–Rabin . . . . . 1

1.4. Probabilidad de error y tipo de algoritmo . . . . . 1

1.5. Modelo de costos y expectativa de candidatos . . . . . 1

**2. Ejercicios resueltos** **2**

2.1. Resuelto 1: división de prueba y Fermat . . . . . 2

2.2. Resuelto 2: Miller–Rabin con traza verificable . . . . . 2

2.3. Resuelto 3: prueba repetida y candidatos . . . . . 3

**3. Ejercicio propuesto 1: biblioteca de primalidad** **4**

3.1. Interfaz y criterio de decisión . . . . . 4

3.2. Casos y trazas: qué demuestra cada salida . . . . . 4

3.3. Validación y alcance . . . . . 5

**4. Ejercicio propuesto 2: comparación experimental** **6**

4.1. Diseño y metodología de medición . . . . . 6

4.2. Comparación de tiempo y operaciones . . . . . 6

4.3. Generación y precribado . . . . . 8

4.4. Testigos, mentirosos fuertes y probabilidad de error . . . . . 9

**5. Respuestas a las preguntas de análisis** **10**

**6. Conclusiones grupales** **12**

# 1. Trabajo preparatorio y fundamentos algebraicos

## 1.1. Unidades módulo $n$, función $\varphi$ y orden

Las unidades $\mathbb{Z}_n^* = \{a \bmod n : \gcd(a,n) = 1\}$ forman un grupo de orden $\varphi(n)$. El orden de $a$ divide $\varphi(n)$ por Lagrange; Euler da $a^{\varphi(n)} \equiv 1 \pmod{n}$. Para un primo $p$, $\varphi(p) = p-1$ y se obtiene Fermat [3, 2]. Si la congruencia falla, $n$ es compuesto; si se cumple, no necesariamente es primo.

## 1.2. División de prueba, pseudoprimos y Carmichael

Si $n = ab$ es compuesto, al menos un factor es $\leq \sqrt{n}$; la división de prueba termina en $\lfloor \sqrt{n} \rfloor$. Para $L$ bits requiere del orden de $2^{L/2}$ divisiones. En cambio, $341 = 11 \cdot 31$ cumple la congruencia de Fermat en base 2: es pseudoprimo para esa base. Los números de Carmichael 561, 1105 y 1729 la cumplen para *toda* base coprima. Según Korselt, son compuestos libres de cuadrados y $(p-1) \mid (n-1)$ para cada primo $p \mid n$ [4].

## 1.3. Exponenciación binaria y Miller–Rabin

La exponenciación binaria calcula $a^e \bmod n$ en $O(\log e)$ productos modulares. Para $n > 2$ impar, Miller–Rabin escribe $n-1 = 2^s d$ ($d$ impar) y examina $a^d, a^{2d}, \ldots, a^{2^{s-1}d}$ módulo $n$. Acepta si aparece 1 al inicio o $-1$ en la cadena. Alcanzar 1 desde una raíz distinta de $\pm 1$ demuestra composición y puede revelar factores mediante $\gcd(x \pm 1, n)$. Una base que rechaza es *testigo*; una que acepta un compuesto, *mentiroso fuerte*. Los cuadrados contabilizados son los ejecutados en la exponenciación inicial y la cadena.

## 1.4. Probabilidad de error y tipo de algoritmo

Para un compuesto impar, a lo sumo un cuarto de las bases válidas son mentirosos fuertes. Con $r$ bases uniformes independientes, la aceptación errónea tiene probabilidad $\leq 4^{-r}$. Miller–Rabin es Monte Carlo de error unilateral: un rechazo es concluyente, una aceptación no. A diferencia de él, un algoritmo Las Vegas siempre responde correctamente, aunque su duración pueda variar. Las semillas fijas reproducen ensayos; la generación criptográfica requiere entropía impredecible y `secrets.SystemRandom` [5].

## 1.5. Modelo de costos y expectativa de candidatos

Por ronda, Fermat y Miller–Rabin realizan $O(L)$ productos modulares; $r$ rondas cuestan $O(rL)$ productos, frente a $O(2^{L/2})$ divisiones de prueba. Con multiplicación escolar, una ronda

requiere $O(L^3)$ operaciones de bit. Entre impares de $L$ bits, la densidad prima aproximada $2/(L\ln 2)$ predice $L\ln 2/2$ candidatos por primo. El precribado ahorra exponenciaciones, no cambia esa densidad inicial.

# 2. Ejercicios resueltos

Los tres ejercicios se ejecutan y verifican con aserciones en `python/ejercicios_resueltos.py`; las salidas siguientes se generan desde ese código.

## 2.1. Resuelto 1: división de prueba y Fermat

La división llega hasta `isqrt(n)`; Fermat comprueba primero $\gcd(a,n)$ y después $a^{n-1}$ mód $n$. Código y salida:

```python
def resuelto_1():
    """División de prueba y contraejemplo al recíproco de Fermat."""
    division_221 = trial_division(221)
    fermat_15 = fermat_round(15, 2)
    fermat_341 = fermat_round(341, 2)
    assert division_221 is False and fermat_15 is False and fermat_341 is True
    return {"221_es_primo": division_221, "15_pasa_fermat": fermat_15,
            "341_pasa_fermat": fermat_341}
```

Listing 1: División de prueba y Fermat: código ejecutable

**Resultado de la ejecución:**

```
>>> resuelto_1()
{'221_es_primo': False, '15_pasa_fermat': False, '341_pasa_fermat': True}
```

$341 = 11 \cdot 31$ pasa Fermat base 2 pese a ser compuesto. Para 561, la base 3 revela un factor común; la base 2 pasa Fermat.

## 2.2. Resuelto 2: Miller–Rabin con traza verificable

Para $221 - 1 = 220 = 2^2 \cdot 55$, la base 2 produce:

$$2^{55} \text{ mód } 221 = 128, \qquad 128^2 \text{ mód } 221 = 30 \neq 220. \tag{1}$$

No aparece $-1 \equiv 220$: 221 es compuesto. La salida incluye parámetros, residuos, motivo y operaciones:

```python
def resuelto_2():
    """Traza verificable de Miller--Rabin para n=221, base a=2."""
    assert decompose(221) == (2, 55)
    passed, detail = miller_rabin_round(221, 2, trace=True)
    assert not passed and detail["residuos"] == [128, 30]
    assert (detail["n"], detail["a"], detail["s"], detail["d"]) == (221, 2, 2, 55)
    return {"pasa": passed, **detail}
```

Listing 2: Miller–Rabin con traza: código ejecutable

**Resultado de la ejecución:**

```
>>> resuelto_2()
{'pasa': False,
 'n': 221,
 'a': 2,
 's': 2,
 'd': 55,
 'residuos': [128, 30],
 'cuadrados_cadena': 1,
 'cuadrados_modulares': 6,
 'productos_modulares': 5,
 'motivo': 'sin_menos_uno'}
```

Las trazas de 341 y 561 se presentan junto con 221 en la Tabla 3; el registro completo está en `python/results/trazas_miller_rabin.csv`.

## 2.3. Resuelto 3: prueba repetida y candidatos

```
def resuelto_3():
    """Candidato reproducible y cinco rondas sobre el primo conocido 1009."""
    rng = Random(2026)
    n = generate_candidate(32, rng)
    assert n.bit_length() == 32 and n % 2 == 1
    report = primality_report(1009, rounds=5, rng=Random(2026))
    assert report["resultado"] == "probable_primo"
    assert report["rondas_ejecutadas"] == 5 and report["cota_error"] == 4.0**-5
    return {"candidato": n, "bits": n.bit_length(), "impar": bool(n % 2),
            "1009_resultado": report["resultado"],
            "rondas_ejecutadas": report["rondas_ejecutadas"],
            "cota_error": report["cota_error"]}
```

Listing 3: Prueba repetida y candidatos: código ejecutable

**Resultado de la ejecución:**

```
>>> resuelto_3()
{'candidato': 2659099673,
 'bits': 32,
 'impar': True,
 '1009_resultado': 'probable_primo',
 'rondas_ejecutadas': 5,
 'cota_error': 0.0009765625}
```

Los bits extremos fijados a 1 garantizan longitud e imparidad. Se prueba 1009, mayor que el límite del precribado, para ejecutar las cinco rondas: se informa “probable primo” y la cota condicional $4^{-5} = 1/1024$. La semilla fija reproduce el ejemplo, pero no es apta para claves.

# 3. Ejercicio propuesto 1: biblioteca de primalidad

## 3.1. Interfaz y criterio de decisión

La biblioteca `python/primality.py` reúne las siete funciones pedidas. Su contrato distingue una prueba concluyente de composición de una aceptación provisional:

Tabla 1: Interfaz pública organizada por función

| Etapa | Funciones | Resultado principal |
|---|---|---|
| Prueba exacta | `trial_division` | Decisión determinista hasta $\lfloor\sqrt{n}\rfloor$; $n<2$ no es primo. |
| Rondas | `fermat_round` | Base válida $2 \le a \le n-2$; $n-1=2^s d$. |
| | `decompose` | La ronda de Miller–Rabin devuelve decisión y detalle: $n, a, s, d$, residuos y contadores. |
| | `miller_rabin_round` | |
| Decisión | `is_probable_prime` | Precribado seguido de rondas; aceptar significa *probable primo*. |
| Generación | `generate_candidate`, | Entero impar de longitud exacta; muestreo hasta superar el precribado y las rondas. |
| | `generate_probable_prime` | |

Se comprueban tipos, bases y descomposiciones antes de operar. El precribado usa primos hasta 997: certifica al candidato que coincide con uno de ellos y descarta sus múltiplos. La exponenciación binaria registra los cuadrados *efectivamente realizados*; no atribuye a `pow()` un conteo interno.

## 3.2. Casos y trazas: qué demuestra cada salida

La Tabla 2 procede de `python/results/casos_conocidos.csv`. “Sí” en Fermat o Miller–Rabin indica que *pasa la base 2*; para $n < 5$ la ronda no aplica. Los cuatro compuestos que pasan Fermat en la tabla son rechazados por Miller–Rabin con esa misma base.

Tabla 2: Casos especiales y pseudoprimos: resultado de las tres pruebas

| $n$ | Clasificación | División | Fermat (2) | MR (2) |
|---|---|---|---|---|
| 0 | ni primo ni compuesto | no | – | – |
| 1 | ni primo ni compuesto | no | – | – |
| 2 | primo | sí | – | – |
| 3 | primo | sí | – | – |
| 25 | compuesto | no | no | no |
| 341 | compuesto | no | sí | no |
| 561 | compuesto | no | sí | no |
| 1105 | compuesto | no | sí | no |
| 1729 | compuesto | no | sí | no |

Tabla 3: Trazas verificables de Miller–Rabin con base 2

| $n$ | $a$ | $s$ | $d$ | Residuos | Decisión |
|---|---|---|---|---|---|
| 221 | 2 | 2 | 55 | [128, 30] | testigo |
| 341 | 2 | 2 | 85 | [32, 1] | testigo |
| 561 | 2 | 4 | 35 | [263, 166, 67, 1] | testigo |

En las trazas de la Tabla 3, generadas desde `python/results/trazas_miller_rabin.csv`, 221 no alcanza $-1$; 341 y 561 alcanzan 1 desde una raíz no trivial. Las tres rondas aportan *testigos de composición*.

## 3.3. Validación y alcance

Las **32 pruebas** de `python/test_primality.py` cubren extremos, cuadrados de primos y bases inválidas; comparan 10 001 enteros con una criba independiente y 1000 aleatorios menores que $10^6$ con división de prueba. Además, `classify_bases` separa testigos y mentirosos fuertes para todas las bases válidas de los compuestos impares $5 \le n < 200$ y de 221, 341 y 561. Los candidatos de 16 a 1024 bits conservan longitud e imparidad.

`primality_report` conserva bases, rondas y operaciones para auditar la decisión. Un testigo certifica composición; si un compuesto impar supera $r$ bases *uniformes e independientes*, la probabilidad de aceptarlo no excede $4^{-r}$. La semilla fija solo reproduce ensayos: sin `rng` se usa `secrets.SystemRandom`, que sí es la fuente prevista para generación segura.

# 4. Ejercicio propuesto 2: comparación experimental

## 4.1. Diseño y metodología de medición

Se estudiaron siete tamaños (16–1024 bits) y se generaron 30 probables primos por tamaño. Para la comparación se prepararon 8 candidatos aceptados y 8 compuestos impares por tamaño. Solo los aceptados de 16 y 32 bits se verificaron también por división de prueba. Las semillas de generación, comparación y simulación fueron `20260701`, `20260702` y `20260703`.

Las mismas entradas y 20 bases preseleccionadas alimentaron Fermat y Miller–Rabin con 1, 5, 10 y 20 rondas; la división se midió solo hasta 32 bits. Se hicieron siete repeticiones por caso con `perf_counter_ns()`, sin selección de bases, impresión ni escritura durante el cronometraje. Se registraron tiempos y operaciones. El precribado se desactivó *solo* para comparar pruebas; en generación permaneció activo.

## 4.2. Comparación de tiempo y operaciones

Las medianas sobre candidatos aceptados (Tabla 4) y la dispersión de siete mediciones de un caso (Tabla 5) no son la misma estadística. Los registros y bases constan en `python/results/comparacion_metodos.csv` y `python/results/entradas_y_bases.csv`.

Tabla 4: Tiempo mediano por prueba sobre probables primos (µs)

| Bits | División | Fermat 20 | MR 1 | MR 5 | MR 20 |
|---|---|---|---|---|---|
| 16 | 17.1 | 173.4 | 14.2 | 56.7 | 209.3 |
| 32 | 6177.6 | 564.2 | 33.1 | 149.2 | 606.2 |
| 64 | – | 1366.2 | 70.8 | 372.6 | 1319.2 |
| 128 | – | 3560.7 | 149.7 | 927.5 | 2947.2 |
| 256 | – | 10164.4 | 434.1 | 2639.2 | 10243.0 |
| 512 | – | 44926.6 | 2300.7 | 10997.1 | 44781.4 |
| 1024 | – | 221081.4 | 10763.3 | 53382.4 | 228012.8 |

Tabla 5: Mínimo, mediana y máximo de siete pruebas sobre una misma entrada ($\mu$s)

| Bits | Método | Mínimo | Mediana | Máximo |
|---|---|---|---|---|
| 16 | División | 17.6 | 17.9 | 19.0 |
| 16 | MR 20 | 207.3 | 217.4 | 241.8 |
| 32 | División | 3746.7 | 4249.9 | 13950.4 |
| 32 | MR 20 | 406.6 | 650.2 | 672.7 |
| 1024 | MR 20 | 209706.7 | 224301.2 | 226631.5 |

**Lectura cuantitativa del costo y error.** Para 1024 bits, la mediana de Miller–Rabin con 20 rondas fue 228012.8 $\mu$s frente a 10763.3 $\mu$s con una ronda (razón 21.2); Fermat con 20 rondas tomó 221081.4 $\mu$s. El compuesto 341 presentó 48 mentirosos fuertes entre 338 bases válidas: $p_{341} = 0{,}1420$. El tiempo medido de Fermat no resuelve sus falsos positivos para Carmichael; cero errores observados tras varias rondas no implica probabilidad cero.

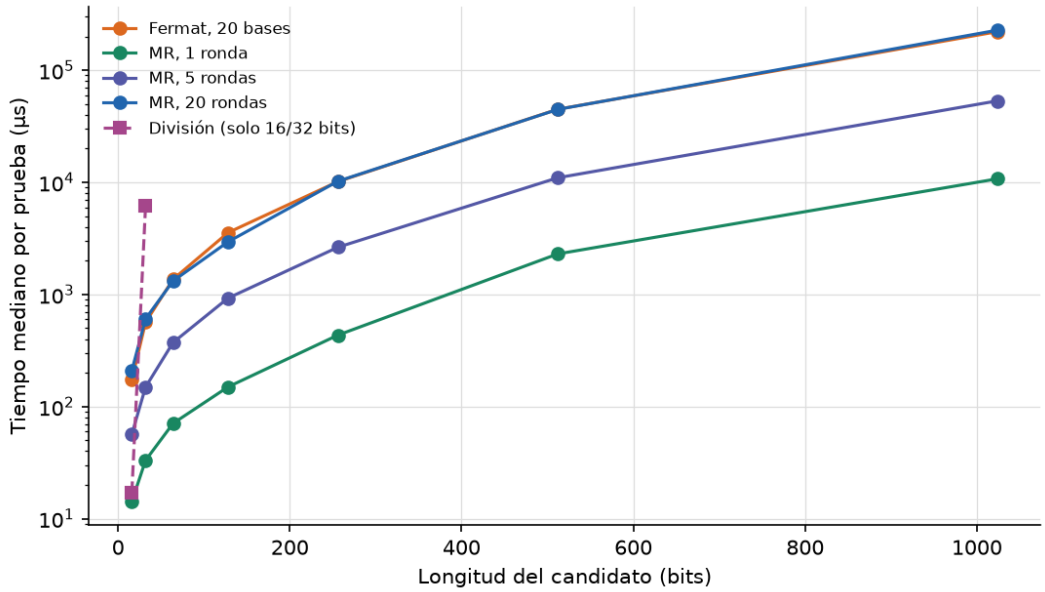



Figura 1: Tiempo mediano por prueba y longitud binaria; escala temporal logarítmica. La división se ejecuta solo hasta 32 bits.

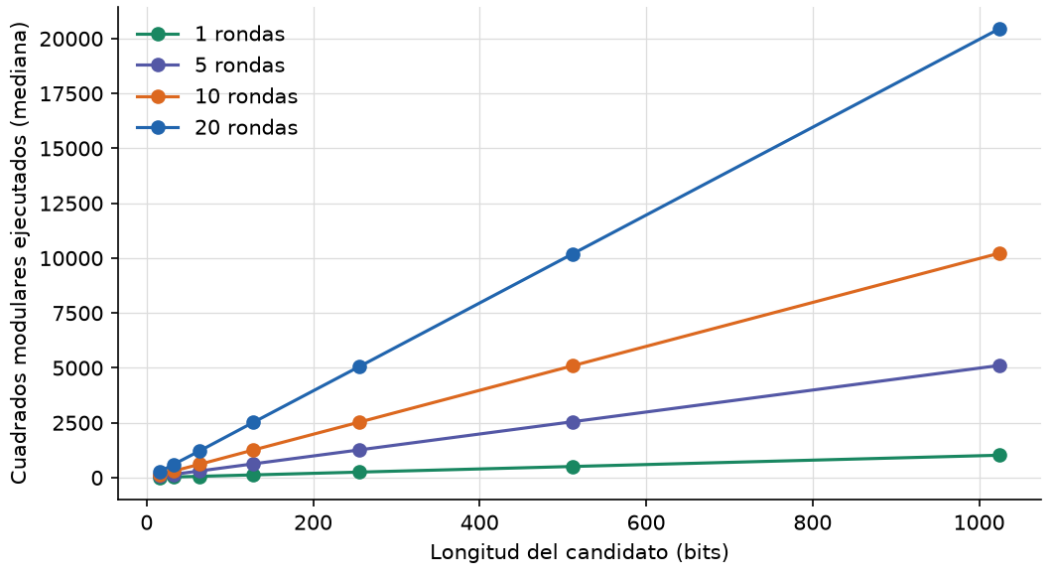



Figura 2: Cuadrados modulares efectivamente realizados por Miller–Rabin con 1, 5, 10 y 20 rondas.

Un testigo detiene las rondas de un compuesto; en un candidato aceptado se completan. El costo por prueba no equivale al costo de buscar un probable primo.

## 4.3. Generación y precribado

La Tabla 6 presenta candidatos, descartes y tiempo; los 210 registros individuales están en `python/results/generacion_210_probables_primos.csv`.

Tabla 6: Generación de 30 probables primos por longitud binaria

| Bits | Candidatos | Media | Mediana | Descartes | Tiempo med. (ms) |
|---|---|---|---|---|---|
| 16 | 169 | 5.63 | 5.0 | 139 | 0.220 |
| 32 | 371 | 12.37 | 10.5 | 310 | 0.720 |
| 64 | 652 | 21.73 | 16.5 | 537 | 1.367 |
| 128 | 970 | 32.33 | 22.5 | 806 | 3.181 |
| 256 | 1840 | 61.33 | 40.5 | 1541 | 10.599 |
| 512 | 5307 | 176.90 | 97.5 | 4445 | 69.182 |
| 1024 | 12870 | 429.00 | 314.5 | 10833 | 802.315 |

**Lectura cuantitativa de la generación.** Para 1024 bits se necesitaron 12870 candidatos para 30 probables primos (media 429.00, mediana 314.5); el precribado descartó 10833 (84.2 %). La referencia asintótica es 354.9 candidatos impares por primo. Para 512 bits, la media observada fue 176.90 frente a 177.4 estimados. Las diferencias con la aproximación pueden reflejar variación muestral; por sí solas no prueban ni descartan errores de primalidad.

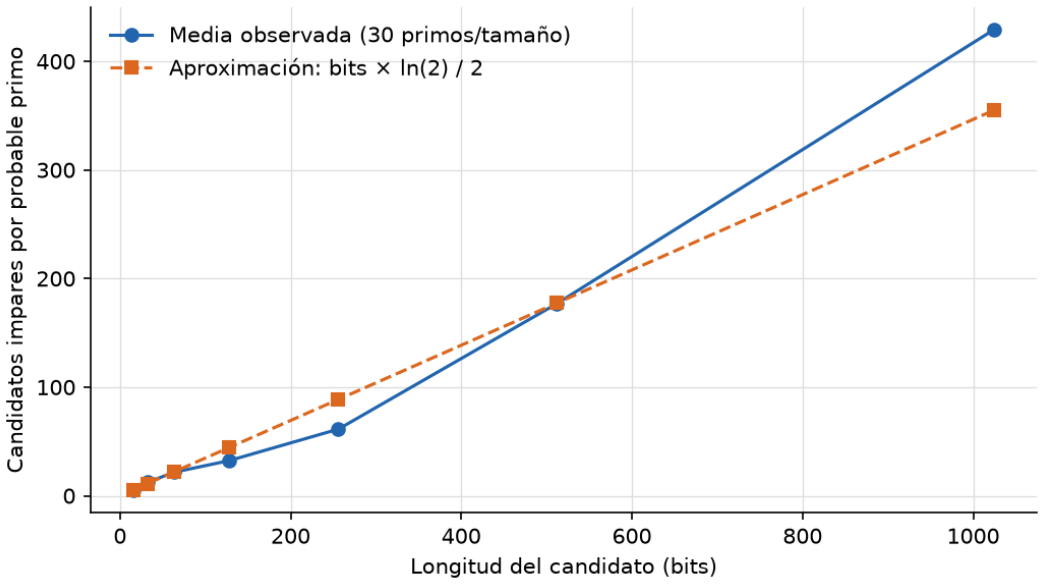



Figura 3: Candidatos impares por probable primo: media de 30 muestras y aproximación $L\ln(2)/2$.

El precribado reduce trabajo, no altera la densidad prima de los candidatos originales.

## 4.4. Testigos, mentirosos fuertes y probabilidad de error

La Tabla 7 cuenta testigos y mentirosos fuertes; `python/results/clasificacion_bases.csv` identifica cada base pequeña. Para un compuesto fijo con fracción $p_n$ de mentirosos, $p_n^r$ es la probabilidad de aceptar $r$ bases uniformes independientes; $4^{-r}$ es solo su cota superior.

La simulación usa bases clasificadas de 25, 91, 121, 341 y 561; no mide tiempo. Cero falsos positivos en una muestra finita no demuestra error imposible. Fermat carece de una cota análoga ante números de Carmichael.

Tabla 7: Testigos y mentirosos fuertes: bases válidas de compuestos seleccionados

| $n$ | Bases | Testigos | Mentirosos | Fracción |
|---|---|---|---|---|
| 25 | 22 | 20 | 2 | 0.0909 |
| 91 | 88 | 72 | 16 | 0.1818 |
| 121 | 118 | 110 | 8 | 0.0678 |
| 221 | 218 | 214 | 4 | 0.0183 |
| 341 | 338 | 290 | 48 | 0.1420 |
| 561 | 558 | 550 | 8 | 0.0143 |

Tabla 8: Aceptación empírica y exacta de compuestos para 3000 ensayos por fila

| $n$ | Rondas | Aceptados | Frecuencia | Exacta $p_n^r$ | Cota $4^{-r}$ |
|---|---|---|---|---|---|
| 25 | 1 | 281 | 0.0937 | 0.0909 | 0.25 |
| 25 | 5 | 0 | 0 | 6.21e-06 | 0.000977 |
| 25 | 20 | 0 | 0 | 1.49e-21 | 9.09e-13 |
| 341 | 1 | 413 | 0.138 | 0.142 | 0.25 |
| 341 | 5 | 0 | 0 | 5.78e-05 | 0.000977 |
| 341 | 20 | 0 | 0 | 1.11e-17 | 9.09e-13 |
| 561 | 1 | 38 | 0.0127 | 0.0143 | 0.25 |
| 561 | 5 | 0 | 0 | 6.06e-10 | 0.000977 |
| 561 | 20 | 0 | 0 | 1.35e-37 | 9.09e-13 |

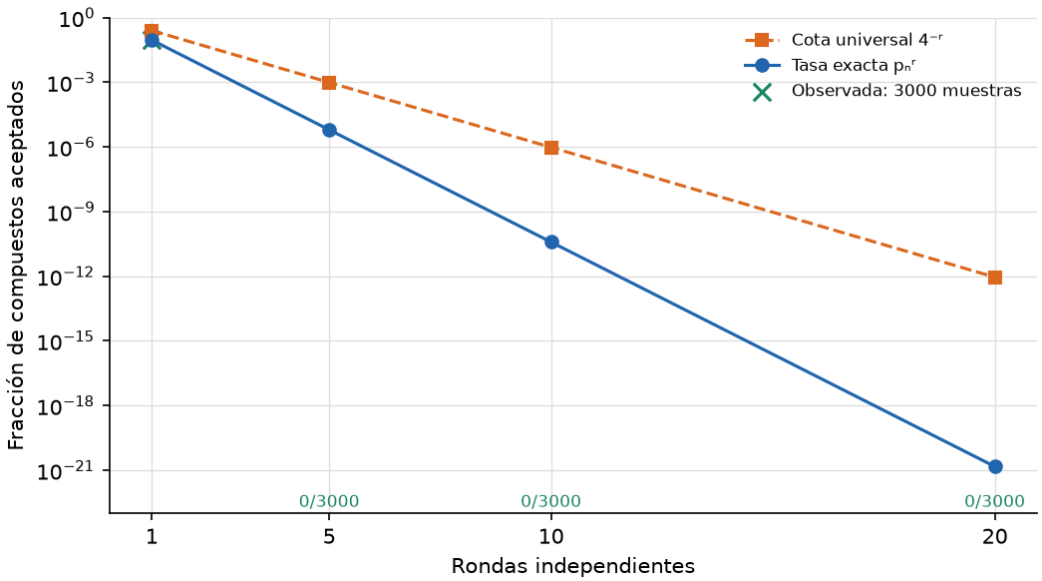



Figura 4: Compuesto 25: tasa exacta $p_n^r$, cota $4^{-r}$ y 3000 ensayos por número de rondas. Los ceros observados se anotan como $0/3000$, no como probabilidades positivas.

# 5. Respuestas a las preguntas de análisis

1. **¿Por qué basta probar divisores hasta $\sqrt{n}$?** Si $n = ab$ y $a > \sqrt{n}$, $b > \sqrt{n}$, entonces $ab > n$, contradicción. Todo compuesto tiene al menos un divisor $\leq \sqrt{n}$.

2. **¿Por qué esa cota es exponencial en bits?** Si $n$ tiene $L$ bits, $\sqrt{n}$ es del orden de $2^{L/2}$: exponencial en $L$, aunque sublineal en $n$.

3. **¿Por qué Fermat puede demostrar composición, pero no primalidad?** Para un primo $p$ y base coprima $a$, Fermat exige $a^{p-1} \equiv 1 \pmod{p}$. Si falla, por contraposición $n$ es compuesto; si pasa, puede ser un pseudoprimo. El ejemplo 341 con base 2 confirma que el recíproco es falso.

4. **¿Qué diferencia hay entre pseudoprimo y número de Carmichael?** Un pseudoprimo pasa Fermat para alguna base coprima; un Carmichael, para todas. 341 pasa base 2; 561 pasa cualquier base coprima, pero la base 3 revela un factor.

5. **¿Por qué Miller–Rabin examina $n-1 = 2^s d$?** Separar la máxima potencia de 2 produce la cadena $a^d, a^{2d}, \ldots, a^{2^{s-1}d}$. En un módulo primo, si el último término es 1, la cadena debe comenzar en 1 o alcanzar $-1$ antes: solo $\pm 1$ son raíces de $x^2 = 1$ en un cuerpo.

6. **¿Qué significa encontrar una raíz no trivial de 1?** Si $x \not\equiv \pm 1 \pmod{n}$ y $x^2 \equiv 1 \pmod{n}$, entonces $n$ es compuesto. Para 341, $x = 32$ y $\gcd(x \pm 1, 341)$ recupera 31 y 11.

7. **¿Qué diferencia hay entre testigo y mentiroso fuerte?** El testigo rechaza un compuesto; el mentiroso fuerte lo acepta. Para 341, la base 2 pasa Fermat, pero es testigo de Miller–Rabin.

8. **¿Por qué el error es unilateral?** Toda base válida hace pasar a un número primo. El rechazo significa compuesto con certeza; la única posible equivocación es aceptar como probable primo un compuesto que superó las bases elegidas.

9. **¿Qué hipótesis permite usar $4^{-r}$?** El número evaluado debe ser compuesto impar; cada base se selecciona uniforme e independientemente entre las bases válidas. La fracción de mentirosos fuertes es $\leq 1/4$ y el evento de $r$ aceptaciones tiene probabilidad $\leq (1/4)^r$. La semilla fija permite reproducir un experimento, no garantiza aleatoriedad criptográfica.

10. **¿Por qué una semilla fija no es apropiada para generar claves?** Permite reconstruir candidatos y bases si se conoce o adivina; se requiere una fuente impredecible como `secrets.SystemRandom`.

11. **¿Cómo influye el precribado?** Descarta múltiplos de primos pequeños y certifica si el candidato coincide con uno de ellos. Ahorra exponenciaciones, sin certificar números grandes que lo superan.

12. **¿Por qué "no observamos errores" no significa "el algoritmo no puede errar"?** Una muestra finita puede omitir las bases mentirosas. La ausencia observada no prueba probabilidad cero; hay que distinguir frecuencia empírica, probabilidad exacta $p_n^r$ para un compuesto dado y cota universal $4^{-r}$.

# 6. Conclusiones grupales

La biblioteca separa tres afirmaciones: un divisor o una base testigo certifican composición; la división de prueba sin divisor certifica primalidad; superar rondas de Miller–Rabin solo indica probable primo. 341, 561, 1105 y 1729 pasan Fermat con base 2, pero Miller–Rabin los rechaza con esa misma base. En 341, la traza $[32, 1]$ revela una raíz no trivial de 1 y permite recuperar los factores 31 y 11.

Las 32 pruebas contrastan 10 001 enteros con una criba independiente, 1000 valores aleatorios menores que $10^6$ con división de prueba y bases pequeñas clasificadas exhaustivamente. Se generaron 210 probables primos entre 16 y 1024 bits; los de 16 y 32 bits se verificaron además por división. A 1024 bits se probaron 12 870 candidatos para obtener 30 aceptados: media de 429 frente a 354,9 de la aproximación teórica. La diferencia puede deberse a la variación muestral.

El rendimiento depende del tamaño y del trabajo realizado. En la comparación local, a 1024 bits la mediana de Miller–Rabin pasó de 10 763,3 $\mu$s con una ronda a 228 012,8 $\mu$s con veinte; un testigo puede detener antes las rondas de un compuesto. El precribado descartó 10 833 candidatos de ese tamaño, evitando pruebas modulares sin alterar la densidad de primos del muestreo.

Por último, cero aceptaciones observadas no significa riesgo nulo: para 25, la frecuencia con cinco rondas fue $0/3000$, pero la probabilidad exacta calculada es $6{,}21 \times 10^{-6}$. La cota $4^{-r}$ presupone bases uniformes independientes; una semilla fija permite reproducir el laboratorio, no generar claves seguras. En conjunto, trazas, conteos y tiempos respaldan decisiones distintas: corrección, incertidumbre y costo deben comunicarse por separado.

# Referencias

[1] H. E. Ugarte Rojas, *Guía de Laboratorio N.º 07: Primalidad, grupos y algoritmos probabilísticos*, Algoritmos Avanzados, UNSAAC, 2026-II.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022.

[3] V. Shoup, *A Computational Introduction to Number Theory and Algebra*, 2.ª ed. Cambridge University Press, 2009.

[4] R. Crandall y C. Pomerance, *Prime Numbers: A Computational Perspective*, 2.ª ed. Springer, 2005.

[5] Python Software Foundation, *secrets — Generate secure random numbers*. https://docs.python.org/3/library/secrets.html.